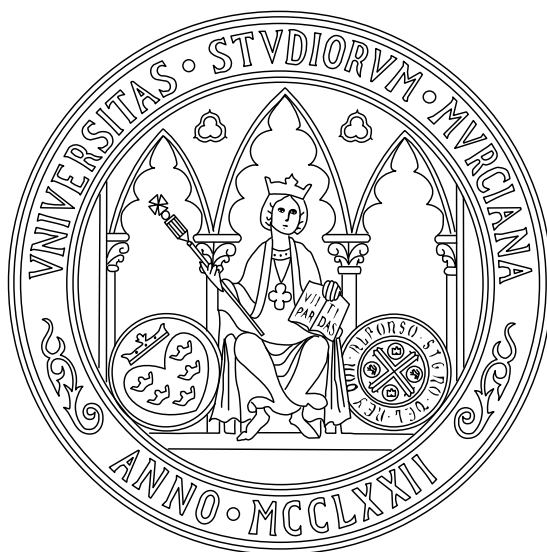




UNIVERSIDAD DE MURCIA
Facultad de Informática

PRÁCTICAS DE
Desarrollo de Aplicaciones Web

4º DE GRADO EN INGENIERÍA INFORMÁTICA
PROFESOR: SALVADOR MANZANARES GUILLÉN
CURSO 2025/2026

Pablo Hernández

DNI

@um.es

Hugo Sánchez Martínez

24450997L

hugo.s.m@um.es

Contenidos

1. Introducción	1
2. Requisitos	1
3. Instalación y ejecución	1
4. Tecnologías utilizadas	2
5. Otros componentes	2
6. Modelado de datos	2
6.1. Diagrama Entidad-Relación	2
6.2. Diagrama de clases	2
7. Descripción de la funcionalidad del sistema	2
7.1. Gestión de acceso y roles	2
7.2. Funcionalidades del administrador	2
7.3. Funcionalidades del usuario vendedor	2
7.4. Funcionalidades del usuario comprador	2
8. Arquitectura de la aplicación	3
8.1. Servicio de usuarios	3
8.2. Servicio de productos	3
8.3. Servicio de compraventa	3
8.4. Servicio de valoraciones	4
8.5. Pasarela de la aplicación	4
9. Diseño e implementación de la interfaz	5
9.1. Sistema de plantillas, páginas, formularios y componentes	5
9.2. Análisis global de las vistas de la aplicación con React Router	5
9.3. Sistema de rejillas con Bootstrap Grid	6
9.4. Autenticación y autorización	7
9.5. Comunicación con el servidor	9
9.6. Otros aspectos y tecnologías	9
10. Conclusiones y valoraciones personales	10
11. Bitácora	10

Índice de figuras

Resumen

Este documento especifica la implementación de las prácticas de la asignatura de Desarrollo de Aplicaciones Web del último curso del grado en Ingeniería Informática de la Universidad de Murcia.

Dicha práctica consiste en el diseño de un frontend para la aplicación desarrollada en la asignatura de Arquitectura del Software del grado.

1. Introducción

Para este ejercicio, es necesario grabar un audio que contenga unos treinta segundos de habla y un segundo de música. Aunque lo más cómodo sería usar un programa con interfaz gráfica como Audacity, podemos complicarnos la vida usando FFmpeg vía línea de comandos:

2. Requisitos

Para garantizar la correcta instalación y puesta en marcha del servicio, el entorno debe cumplir con los siguientes requisitos previos:

- **Node.js** (v20.20 o superior): Para ejecutar el cliente. [Descargar](#).
- **Docker**: Para levantar los servicios del servidor. [Descargar](#).

El resto de las dependencias específicas del proyecto serán descargadas y resueltas automáticamente por las herramientas mencionadas durante el proceso de instalación.

3. Instalación y ejecución

El proceso de despliegue de la aplicación se divide en dos partes principales: los servicios del backend y la interfaz del frontend.

Los servicios correspondientes al primero están contenerizados para asegurar un entorno de ejecución consistente. Dicha infraestructura se despliega y orquesta mediante Docker Compose. Para levantar estos servicios, basta con ejecutar el siguiente comando desde el directorio `backend/`:

```
1 $ docker compose up -d
```

Para detener el servidor:

```
1 $ docker compose down
```

Para levantar el cliente (frontend), es necesario, desde su carpeta correspondiente `frontend/`, instalar las dependencias y arrancar en modo de desarrollo con:

```
1 $ npm install && npm run dev
```

4. Tecnologías utilizadas

Para el desarrollo de la interfaz de usuario, se ha empleado **React** como biblioteca principal. Este *framework* permite la construcción de una arquitectura robusta basada en componentes reutilizables. Complementariamente, el diseño visual y la estructuración de la interfaz se han implementado mediante la librería **React-Bootstrap**, que integra componentes estilizados de Bootstrap directamente en el ecosistema de React.

El ecosistema de desarrollo se sustenta sobre un entorno de ejecución **Node.js**, que proporciona la infraestructura necesaria para la gestión de dependencias. Como herramienta de construcción y empaquetado, se ha seleccionado **Vite**.

Para lograr un comportamiento correspondiente al de una *Single Page Application* (SPA), la navegación interna ha sido gestionada mediante **React Router**. Específicamente, se ha implementado su modo de enrutamiento declarativo.

Finalmente, para la gestión del ciclo de vida del código fuente, se ha utilizado el sistema de control de versiones **Git**, que proporciona un historial de cambios y un entorno de colaboración con trazabilidad.

5. Otros componentes

6. Modelado de datos

6.1. Diagrama Entidad-Relación

6.2. Diagrama de clases

7. Descripción de la funcionalidad del sistema

7.1. Gestión de acceso y roles

7.2. Funcionalidades del administrador

7.3. Funcionalidades del usuario vendedor

7.4. Funcionalidades del usuario comprador

8. Arquitectura de la aplicación

El backend de la aplicación corresponde al sistema distribuido desarrollado durante las prácticas de la asignatura de Arquitectura del Software. Es una arquitectura basada en microservicios, donde cada componente es responsable de un dominio de negocio específico. Los microservicios se comunican usando interfaces como Retrofit y un broker de mensajes RabbitMQ. Toda la comunicación se centraliza usando el microservicio *pasarela*, que actúa de API Gateway para el resto.

A continuación se detallan brevemente los diferentes servicios que componen la arquitectura y los *endpoints* que exponen en sus controladores.

8.1. Servicio de usuarios

Este servicio se encarga de la gestión del ciclo de vida de los usuarios en el sistema. Permite dar de alta a nuevos usuarios, así como consultar y actualizar sus datos personales.

HTTP	Endpoint
GET	/usuarios
POST	/usuarios
GET	/usuarios/{id}
PUT	/usuarios/{id}

Cuadro 1: Endpoints expuestos por el servicio de usuarios.

8.2. Servicio de productos

El servicio de productos gestiona el catálogo de artículos disponibles en la plataforma. Entre su funcionalidad, encontramos la creación de nuevos productos, la consulta y modificación de los mismos, la gestión de su estado de recogida, así como el registro de visualizaciones de cada producto.

HTTP	Endpoint
GET	/productos
POST	/productos
GET	/productos/{id}
PUT	/productos/{id}
PUT	/productos/{id}/recogida
POST	/productos/{id}/visualizaciones
GET	/productos/historial/{mes}/{anio}

Cuadro 2: Endpoints expuestos por el servicio de productos.

8.3. Servicio de compraventa

Para gestionar las transacciones entre compradores y vendedores, se define un servicio dedicado, que permite registrar nuevas operaciones de compraventa y recuperar el historial de transacciones tanto desde la perspectiva del comprador como del vendedor.

HTTP	Endpoint
<i>GET</i>	/compraventas
<i>POST</i>	/compraventas
<i>GET</i>	/compraventas/{id}
<i>GET</i>	/compraventas/compras/{idComprador}
<i>GET</i>	/compraventas/ventas/{idVendedor}

Cuadro 3: Endpoints expuestos por el servicio de compraventas.

8.4. Servicio de valoraciones

Este servicio permite a los usuarios calificar y dejar comentarios sobre otros usuarios tras haber finalizado una transacción. Proporciona operaciones para registrar nuevas valoraciones y consultarlas en base al vendedor o al comprador.

HTTP	Endpoint
<i>POST</i>	/api/valoraciones
<i>GET</i>	/api/valoraciones/{id}
<i>GET</i>	/api/valoraciones/vendedor/{vendedorId}
<i>GET</i>	/api/valoraciones/comprador/{compradorId}

Cuadro 4: Endpoints expuestos por el servicio de valoraciones.

8.5. Pasarela de la aplicación

La pasarela (API Gateway) actúa como punto de entrada único para las peticiones al servidor. Además de enrutar las llamadas a los microservicios correspondientes, se encarga la autenticación y de la gestión de sesiones mediante el uso de tokens JWT. Sin embargo, la autorización de roles corresponde a cada microservicio.

HTTP	Endpoint
<i>POST</i>	/login
<i>POST</i>	/refresh
<i>POST</i>	/logout

Cuadro 5: Endpoints de autenticación expuestos por la pasarela.

9. Diseño e implementación de la interfaz

9.1. Sistema de plantillas, páginas, formularios y componentes

El diseño de la aplicación se ha conseguido siguiendo un enfoque modular, haciendo uso de componentes y *layouts* (plantillas de diseño), aprovechando las capacidades de React y React Router.

Los componentes representan piezas de interfaz independientes y reutilizables, como pueden ser el `Header`, el `Footer` o el menú lateral `Menu`. Aislar la lógica de estos componentes a ficheros dedicados ha facilitado enormemente el desarrollo gracias a la escalabilidad que proporciona, y ha resultado vital para el mantenimiento del código.

Para estructurar las vistas, se han definido diferentes plantillas que actúan como envoltorios para las rutas. Por ejemplo, el `MainLayout` define la estructura base con un encabezado, un contenedor central y un pie de página. Como cada página que emplee esta plantilla renderiza un componente principal distinto, se hace uso del elemento `<Outlet />` de React Router.

Para el panel, se han podido juntar las vistas de administración de usuario gracias a definir un `PanelLayout`. Este layout divide la vista en dos columnas: una barra lateral fija que contiene el componente `Menu` y una sección principal donde se carga el contenido específico mediante su propio `<Outlet />`.

9.2. Análisis global de las vistas de la aplicación con React Router

La navegación de la aplicación en el lado del cliente se gestiona utilizando **React Router**. Esta biblioteca permite definir rutas de distinta forma (*Data*, *Framework* y *Declarative*). A pesar de sus limitaciones frente al resto, se ha escogido el enfoque declarativo debido a su sencillez y porque es el usado en las prácticas.

De esta forma, podemos asociar cada URL de navegación con la jerarquía de componentes que debe renderizarse, indicando, en una misma porción de código, todo lo que necesitamos para la correcta navegación.

En el componente principal, se emplea el contenedor `<Routes>` para agrupar todas las rutas de la aplicación, anidándolas bajo distintos envoltorios (plantillas) junto con reglas de autorización:

```
1 // App.jsx
2 <Routes>
3   <Route element={MainLayout />>
4     <Route path="/" element={Inicio /> />
5     <Route path="/buscar" element={Buscar /> />
6   </Route>
7
8   <Route element={AuthLayout />>
9     <Route path="/login" element={Login /> />
10    <Route path="/signup" element={Register /> />
11  </Route>
12  ...
13 </Routes>
```

Como se observa en el código, el enrutamiento se organiza lógicamente estructurando las distintas páginas como componentes hijos de su plantilla correspondiente. A continuación muestra como se agrupan cada una de las vistas:

- `MainLayout` : Plantilla base con encabezado y pie de página. Engloba:
 - `/` : Vista principal de aterrizaje (`Inicio`).
 - `/buscar` : Motor de búsqueda y catálogo general de productos (`Buscar`).

- **AuthLayout** : Plantilla más simple que la anterior, que recoge los formularios de autenticación. Engloba:
 - `/login` : Formulario de inicio de sesión (`Login`).
 - `/signup` : Formulario de creación de cuenta (`Register`).
- **PanelLayout** : Plantilla de vistas privadas. Están protegidas mediante el componente `RequireAuth` , y se dividen en:
 - Panel de usuarios. Requieren el rol `Roles.USUARIO` .
 - `/panel/mi-cuenta` : Gestión de los datos del perfil (`MiCuenta`).
 - `/panel/productos` : Gestión de los artículos puestos a la venta (`MisProductos`).
 - Panel de administradores. Requieren el rol `Roles.ADMIN` .
 - `/panel/admin/usuarios` : Vista de administración de usuarios.
 - `/panel/admin/compraventas` : Vista de administración de transacciones.
- **Otras vistas**: También se incluyen páginas adicionales que se invocan ante errores de autorización o de páginas no encontradas.
 - `/404` : Vista a la que se redirige cuando la ruta solicitada no existe (`Error404`).
 - `/unauthorized` : Vista mostrada cuando el usuario carece de permisos de rol suficientes para un recurso específico (`Unauthorized`).

9.3. Sistema de rejillas con Bootstrap Grid

Para la visualización de los productos en las vistas de búsqueda e inicio de la aplicación, se ha implementado un sistema de rejilla adaptable y dinámico en el componente `GridProductos` . Este componente aprovecha las ventajas de los componentes `Container` , `Row` y `Col` de la biblioteca React Bootstrap.

El *responsiveness* se ha logrado usando puntos de ruptura (*breakpoints*) para los distintos tamaños de pantalla. Así, por ejemplo, tendremos filas de cuatro productos en pantallas `xxl` ($\geq 1400\text{ px}$) o filas de un único producto en pantallas `xs` o `sm` ($< 768\text{ px}$). De esta forma, la cantidad de tarjetas de productos por fila varía dinámicamente en función del ancho de la pantalla del dispositivo.

```

1  return (
2    <Container fluid className={className}>
3      <Row xs={1} sm={1} md={2} lg={2} xl={3} xxl={4} className="g-4">
4        {nuevoProductoCard ? (
5          <Col key="lead-card">{nuevoProductoCard}</Col>
6        ) : null}
7        {PRODUCTOS.map((producto) => (
8          <Col key={producto.id}>
9            <Card className="h-100 rounded-5 overflow-hidden shadow-sm">
10              ...
11            </Card>
12          </Col>
13        ))}
14      </Row>
15    </Container>
16  );

```

De forma adicional, se hace uso de la clase de utilidad de Bootstrap `g-4` (*gutter-4*) que define un *padding* horizontal y vertical para crear separación entre las tarjetas.

9.4. Autenticación y autorización

Las aplicaciones React mantienen el estado de sus componentes únicamente mientras la aplicación permanece en ejecución. Cuando la página se recarga o el usuario inicia una nueva sesión en el navegador, dicho estado se pierde. Por este motivo, las aplicaciones que requieren autenticación deben utilizar mecanismos de almacenamiento persistente que permitan conservar la información de sesión. En React existen, principalmente, tres zonas en las que guardar información:

- **In-memory state:** Guarda el estado de los componentes. Se pierde al renderizar otros componentes, por ejemplo, refrescando la página.
- **Local Storage:** Es un sistema de información persistente del navegador. Guarda información entre las distintas sesiones.
- **Cookies:** Son pequeños fragmentos de información almacenados por el navegador. A diferencia del anterior, se envían automáticamente al servidor en cada petición HTTP.

Una primera idea sería tener un token JWT suministrado por el backend al acceder a `/login`. Este token no se podría almacenar en memoria, ya que se perdería, por lo que habría que almacenarlo en el Local Storage del navegador. Este enfoque conlleva diversos problemas de seguridad, ya que el *Local Storage* es accesible directamente desde JavaScript. Esto implica que, en caso de una vulnerabilidad de tipo **Cross-Site Scripting (XSS)**, un atacante podría obtener el token y suplantar la identidad del usuario. Por este motivo, almacenar tokens de autenticación en el navegador sin protección adicional no es una buena práctica.

Una alternativa más robusta consistiría en el uso de una `cookie` configurada con el atributo `HttpOnly`, que bloquea el acceso de JavaScript y soluciona el problema mencionado. Esta opción es la recomendada por la asignatura de Arquitectura del Software. Pero presenta un problema evidente: durante el ciclo de vida de la cookie, estamos confiando plenamente en el usuario. Si la duración de vida es corta, implementaríamos un mecanismo para actualizar el token con un endpoint `/refresh`, pero nunca llegaríamos a volver a solicitar las credenciales al usuario. Si la duración de vida es larga, tendríamos que confiar en el usuario durante dicho período de tiempo. Guardar las credenciales del usuario de forma persistente tampoco es una opción viable. Además, somos vulnerables a un segundo tipo de ataque, **Cross-Site Request Forgery (CSRF)**, pues estamos enviando la cookie en cada petición.

En la actualidad, existe una solución híbrida que combina ambos conceptos para equilibrar la protección contra ataques XSS y CSRF con la flexibilidad que exige una API moderna. Dicho estándar propone un enfoque con dos tokens:

- **Access Token:** De corta duración (10 o 15 minutos), es generado por el backend al iniciar sesión. El cliente (en nuestro caso, la aplicación React) almacena este token únicamente en memoria y lo envía en la cabecera `Authorization: Bearer <token>` en cada petición a la API. Al no estar en el almacenamiento persistente, se la exposición frente a un ataque XSS, y al no ser una cookie, es inmune al CSRF.
- **Refresh Token:** De larga duración (unos 7 días), también es generado por el backend y establecido al iniciar sesión. Cuando el token de acceso expira y se pierde de la memoria, el cliente realiza una petición a `/refresh` para renovar dicho token.

En el lado del cliente, se ha desarrollado un componente interceptor `RequireAuth`, que envuelve las rutas protegidas y verifica si el usuario autenticado posee al menos uno de los roles permitidos (`allowedRoles`) especificados en la definición del enrutador de React Router. Si el usuario cuenta con los permisos necesarios, se renderiza la vista solicitada (componente `<Outlet />`). En caso de tener los permisos de rol necesarios, se le redirige a `/unauthorized`. Si directamente, el usuario no estaba “logueado”, se le redirige a la vista para iniciar sesión.

```

1 // RequireAuth.jsx
2 const RequireAuth = ({ allowedRoles }) => {
3   const { auth } = useAuth();
4   const location = useLocation();
5
6   return auth?.roles?.find((role) => allowedRoles?.includes(role)) ? (
7     <Outlet />
8   ) : auth?.usuario ? (
9     <Navigate to="/unauthorized" state={{ from: location }} replace />
10   ) : (
11     <Navigate to="/login" state={{ from: location }} replace />
12   );
13 };

```

9.4.1. Gestión y ciclo de vida de los tokens

La sincronización de las peticiones HTTP privadas y la actualización de las credenciales expiradas se lleva a cabo mediante interceptores de Axios y hooks personalizados:

- **Hook `useRefreshToken`** : Expone una función para solicitar la renovación del token de acceso. Realiza una petición al endpoint de renovación (`/refresh`), el cual lee el Refresh Token, enviado automáticamente por el navegador.
- **Hook `useApiPrivate`** : Configura una instancia de Axios dedicada a realizar peticiones HTTP autenticadas. Esta instancia incluye dos interceptores:
 - **Interceptor de petición (*Request Interceptor*)**: Es necesario para inyectar de forma automática la cabecera `Authorization: Bearer <token>` con el Access Token actual antes de que la petición salga al servidor.
 - **Interceptor de respuesta (*Response Interceptor*)**: Es posible que el Access Token haya caducado, pues su *lifespan* es bastante corto. El backend informará con un error HTTP 401 Unauthorized. Es aquí donde éste interceptor actúa, iniciando el proceso de renovación llamando al hook definido anteriormente como `useRefreshToken`, actualizando la cabecera de la petición original con el nuevo Access Token y volviéndola a intentar (una única vez).

Se puede dar el caso en el que, o bien por caducidad o por invalidez, el Refresh Token almacenado en la cookie del navegador ya no sea válido. En este caso, el frontend debe darse cuenta de que el usuario ha perdido su sesión persistente, y que debe volver a identificarse con sus credenciales en el servidor. El interceptor analiza la respuesta ante un intento de renovación, y redirige al usuario forzosamente al formulario de `/login` en caso de un Refresh Token inválido.

```

1 // useApiPrivate.jsx
2 const requestId = instancePrivate.interceptors.request.use(
3   (config) => {
4     config.headers['Authorization'] ??= `Bearer ${authRef.current?.accessToken}`;
5     return config;
6   },
7   (error) => Promise.reject(error)
8 );
9
10 const responseId = instancePrivate.interceptors.response.use(
11   (response) => response,
12   async (error) => {
13     const prevReq = error?.config;
14
15     if (error?.response?.status === 401 && !prevReq?.sent) {
16       prevReq.sent = true; // un único intento
17
18       try {

```

```

19     refreshPromise ??= refreshRef.current().finally(() => refreshPromise = null);
20     prevReq.headers['Authorization'] = `Bearer ${await refreshPromise}`;
21     return instancePrivate(prevReq);
22   } catch { // limpiar auth y redirigir a /login
23     setAuth({}); navigate('/login', { replace: true });
24   }
25 }
26
27 return Promise.reject(error);
28 }
29 );

```

9.5. Comunicación con el servidor

9.6. Otros aspectos y tecnologías

Para complementar el ecosistema de desarrollo y enriquecer la experiencia final del usuario, se han adoptado herramientas y configuraciones adicionales:

A medida que la jerarquía de carpetas se vuelve más profunda, vimos que una buena práctica en estos entornos es usar **rutas absolutas** en vez de rutas relativas [1]. Para implementarlo, se han configurado rutas absolutas mediante el uso de alias (`@/`). Esto ha requerido configurar Vite en el archivo `vite.config.js`, ya que es el *bundler* empleado, y definir la resolución de los distintos directorios en `jsconfig.json`, aunque este último es solo para el analizador estático ESLint.

También se han usado librerías externas de código abierto para mejorar la experiencia de usuario. Una de ellas es el **DatePicker** de [Tempus Dominus](#) o el **International Telephone Input** para números de teléfono [Intl-Tel-Input](#). Estos componentes han resultado un acierto para facilitar la validación de dichos campos e introducir restricciones que, de otras formas, habría que configurar manualmente.

10. Conclusiones y valoraciones personales

El desarrollo de esta práctica ha resultado muy útil para

11. Bitácora

A continuación, se comenta la distribución del trabajo entre las clases prácticas presenciales y el trabajo autónomo. El control de horas es una aproximación cercana a la duración real de cada actividad.

Actividad	Presencial	Trabajo autónomo
<i>Análisis</i>	2	2
<i>Desarrollo</i>	0	3
<i>Documentación</i>	0	2
Total	2 horas	7 horas

Cuadro 6: Distribución de horas entre actividades presenciales y trabajo autónomo.

Referencias

- [1] SalteadorNeo. Alias y rutas absolutas en vite. <https://salteadorneo.dev/blog/alias-rutas-vite/>. Accedido el: 28/05/2026.